

TP n° 2 : Requêtes, dépendances fonctionnelles et normalisation

Remarques En cas de difficulté à vous connecter rappez-vous de la section « Préparation de l'environnement de travail » de la série de TP n° 1.

Dans le précédent TP, vous avez utilisé *SQLPLUS*. Je vous invite, au cours de ce TP à découvrir le client graphique *SQLDeveloper*, que vous pouvez lancer, en exécutant la commande suivante :

```
1 syoucef$ sqldeveloper
```

Vous déposez le travail réalisé sur l'ENT.

Exercice n° 1

1. Afficher le nom du département qui a le budget le plus élevé.
2. Afficher les salaires et les noms des enseignants qui gagnent plus que le salaire moyen.
3. Pour chaque enseignant, afficher tous les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant ainsi que le nombre total de cours suivis par chaque étudiant, en utilisant la clause *HAVING*.
4. Pour chaque enseignant, afficher tous les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant ainsi que le nombre total de cours suivis par chaque étudiant, sans utiliser la clause *HAVING*.
5. Afficher les identifiants et les noms des étudiants qui n'ont pas suivi de cours avant 2010.
6. Afficher tous les enseignants dont les noms commencent par E.
7. Afficher les salaires et les noms des enseignants qui perçoivent le quatrième salaire le plus élevé.
8. Afficher les noms et les salaires des trois enseignants qui perçoivent les salaires les moins élevés. Les afficher par ordre décroissant.
9. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause *IN*.
10. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause *SOME*.
11. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la jointure naturelle (*NATURAL INNER JOIN*).
12. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause *EXISTS*.
13. Afficher toutes les paires des étudiants qui ont suivi au moins un cours ensemble.
14. Afficher pour chaque enseignant, qui a effectivement assuré un cours, le nombre total d'étudiants qui ont suivi ses cours. Si un étudiant a suivi deux cours différents avec le même enseignant, on le compte deux fois. Trier le résultat par ordre décroissant.
15. Afficher pour chaque enseignant, même s'il n'a pas assuré de cours, le nombre total d'étudiants qui ont suivi ses cours. Si un étudiant a suivi deux fois un cours avec le même enseignant, on le compte deux fois. Trier le résultat par ordre décroissant.
16. Pour chaque enseignant, afficher le nombre total de *grades A* qu'il a attribué.
17. Afficher toutes les paires enseignants et élèves ou un élève a suivi le cours de l'enseignant, ainsi que le nombre de fois que cet élève a suivi un cours dispensé par cet enseignant.
18. Afficher toutes les paires enseignant- élève ou un élève a suivi au moins deux cours dispensés par l'enseignant en question.

ID	name
19991	Brandt
23121	Chavez
55739	Sanchez
70557	Snow

4 rows in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

ID	name	dept_name	salary
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00

2 rows in set (0.000 sec)

```

+-----+
| dept_name |
+-----+
| Finance   |
+-----+

```

1 row in set (0.023 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

```

+-----+
| name   | salary |
+-----+
| Einstein | 95000.00 |
| Brandt  | 92000.00 |
| Wu      | 90000.00 |
| Gold    | 87000.00 |
| Singh   | 80000.00 |
| Kim     | 80000.00 |
| Katz     | 75000.00 |
+-----+

```

7 rows in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

```

+-----+
| instructor_id | instructor_name | student_id | student_name | nb_cours_suivis_avec_cet_enseignant |
+-----+
| 10101         | Srinivasan      | 12345      | Shankar      | 3 |
+-----+

```

1 row in set (0.002 sec)

instructor_id	instructor_name	student_id	student_name	nb_fois
83821	Brandt	76543	Brown	1
83821	Brandt	12345	Shankar	1
83821	Brandt	54321	Williams	1
76766	Crick	98988	Tanaka	2
22222	Einstein	44553	Peltier	1
32343	El Said	19991	Brandt	1
45565	Katz	45678	Levy	2
98345	Kim	76653	Aoi	1
15151	Mozart	55739	Sanchez	1
10101	Srinivasan	98765	Bourikas	2
10101	Srinivasan	76543	Brown	1
10101	Srinivasan	45678	Levy	1
10101	Srinivasan	12345	Shankar	3
10101	Srinivasan	54321	Williams	1
10101	Srinivasan	00128	Zhang	2
12121	Wu	23121	Chavez	1

16 rows in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

instructor_id	instructor_name	student_id	student_name	nb_fois
76766	Crick	98988	Tanaka	2
45565	Katz	45678	Levy	2
10101	Srinivasan	12345	Shankar	3
10101	Srinivasan	98765	Bourikas	2
10101	Srinivasan	00128	Zhang	2

instructor_id	instructor_name	nb_grades_A
10101	Srinivasan	4
83821	Brandt	2
76766	Crick	1

3 rows in set (0.000 sec)

instructor_id	instructor_name	total_inscriptions
10101	Srinivasan	10
83821	Brandt	3
76766	Crick	2
45565	Katz	2
22222	Einstein	1
32343	El Said	1
98345	Kim	1
15151	Mozart	1
12121	Wu	1

9 rows in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

instructor_id	instructor_name	total_inscriptions
10101	Srinivasan	10
83821	Brandt	3
76766	Crick	2
45565	Katz	2
22222	Einstein	1
32343	El Said	1
98345	Kim	1
15151	Mozart	1
12121	Wu	1
58583	Califieri	0
33456	Gold	0
76543	Singh	0

12 rows in set (0.000 sec)

name
Bourikas
Brown
Levy
Peltier
Shankar
Williams
Zhang

7 rows in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

student1_id	student1_name	student2_id	student2_name
00128	Zhang	12345	Shankar
00128	Zhang	45678	Levy
00128	Zhang	54321	Williams
00128	Zhang	76543	Brown
00128	Zhang	98765	Bourikas
12345	Shankar	45678	Levy
12345	Shankar	54321	Williams
12345	Shankar	76543	Brown
12345	Shankar	98765	Bourikas
45678	Levy	54321	Williams
45678	Levy	76543	Brown
45678	Levy	98765	Bourikas
54321	Williams	76543	Brown
54321	Williams	98765	Bourikas
76543	Brown	98765	Bourikas

name
Bourikas
Brown
Levy
Peltier
Shankar
Williams
Zhang

7 rows in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

name
Bourikas
Brown
Levy
Peltier
Shankar
Williams
Zhang

Exercice n° 2

Donner la forme la plus avancée des schémas de relations suivants, munis de l'ensemble de dépendances fonctionnelles F . Si une relation n'est pas normalisée, la décomposer pour atteindre la forme normale la plus avancée.

1. $R(A, B, C)$ et $F = \{A \twoheadrightarrow B; B \twoheadrightarrow C\}$.

Forme normale : 3NF (dépendance transitive $B \rightarrow C$)

Décomposition en 3NF :

$R_1(A, B)$ avec $F_1 = \{A \rightarrow B\}$

$R_2(B, C)$ avec $F_2 = \{B \rightarrow C\}$

2. $R(A, B, C)$ et $F = \{A \twoheadrightarrow C; A \twoheadrightarrow B\}$.

Forme normale : BCNF (A est clé candidate)

Décomposition : Déjà en BCNF

3. $R(A, B, C)$ et $F = \{A, B \twoheadrightarrow C; C \twoheadrightarrow B\}$.

Forme normale : 3NF ($C \rightarrow B$, C n'est pas super-clé)

Décomposition en BCNF :

$R_1(C, B)$ avec $F_1 = \{C \rightarrow B\}$

$R_2(A, C)$ avec $F_2 = \{\}$

Exercice n° 3

1. Soit une relation $R(A, B, C, D, E)$ et un ensemble de dépendances fonctionnelles $F = \{A \twoheadrightarrow B, C; C, D \twoheadrightarrow E; B \twoheadrightarrow D; E \twoheadrightarrow A\}$. D'édire au moins 16 d'ependances fonctionnelles, en utilisant le syst`eme d'Armstrong (règles de réflexivité, augmentation et transitivité(e)).

Déduction de dépendances fonctionnelles (16 minimum) :

Par réflexivité :

- $A, B \rightarrow A$; $A, B \rightarrow B$; etc.

Par augmentation :

- $A \rightarrow B, C \Rightarrow A, D \rightarrow B, C, D$
- $B \rightarrow D \Rightarrow A, B \rightarrow A, B, D$

Par transitivité :

- $A \rightarrow B, C$ et $B \rightarrow D \Rightarrow A \rightarrow B, C, D$
- $A \rightarrow B, C, D$ et $C, D \rightarrow E \Rightarrow A \rightarrow B, C, D, E$
- $E \rightarrow A$ et $A \rightarrow B, C, D, E \Rightarrow E \rightarrow B, C, D, E$

Fermeture de F (F^+) : Ensemble de toutes les DF dérivables.

2. Soit une relation $R(A, B, C, D, E, F)$ et un ensemble de dépendances fonctionnelles
 $F = \{A \twoheadrightarrow B, C, D; B, C \twoheadrightarrow D, E; B \twoheadrightarrow D; D \twoheadrightarrow A\}$.

(a) Calculer la fermeture de l'attribut B et de l'ensemble $\{A, B\}$

attributs : B^+ :

- $B \twoheadrightarrow D \Rightarrow B^+ = \{B, D\}$
- $D \twoheadrightarrow A \Rightarrow B^+ = \{A, B, D\}$
- $A \twoheadrightarrow B, C, D \Rightarrow B^+ = \{A, B, C, D\}$
- $B, C \twoheadrightarrow D, E \Rightarrow B^+ = \{A, B, C, D, E\}$

Résultat : $B^+ = \{A, B, C, D, E\}$

$\{A, B\}^+$:

- $A \twoheadrightarrow B, C, D \Rightarrow \{A, B, C, D\}$
- $B \twoheadrightarrow D$ (déjà)
- $C \rightarrow ?$ (pas de DF directe)
- $D \twoheadrightarrow A$ (déjà)
- $B, C \twoheadrightarrow D, E \Rightarrow \{A, B, C, D, E\}$

Résultat : $\{A, B\}^+ = \{A, B, C, D, E\}$

(b) Montrer que $\{A, F\}$ est une super clé.

Calcul de $\{A, F\}^+$:

- $A \twoheadrightarrow B, C, D \Rightarrow \{A, B, C, D, F\}$
- $B \twoheadrightarrow D$ (déjà)
- $B, C \twoheadrightarrow D, E \Rightarrow \{A, B, C, D, E, F\}$

Résultat : $\{A, F\}^+ = R$, donc $\{A, F\}$ est une super-clé.

(c) Est-elle en BCNF (Boyce–Codd Normal Form) ? Si R n'est pas en BCNF, la décomposition pour atteindre cette forme normale.

Vérification : Toute DF $X \twoheadrightarrow Y$ doit avoir X comme super-clé.

- $A \twoheadrightarrow B, C, D$: A n'est pas super-clé ($A^+ = \{A, B, C, D\}$) $\rightarrow$ violation BCNF

Décomposition BCNF :

- Pour $A \twoheadrightarrow B, C, D$, décomposer en :
 - $R_1(A, B, C, D)$
 - $R_2(A, E, F)$

- Vérifier chaque sous-relation.

3. Soit une relation $R(A, B, C, D, E)$ que l'on décompose en deux relations

(a) $R_1(A, B, C)$ et $R_2(A, D, E)$. Montrer que cette décomposition est sans perte d'information. L'intersection des attributs est $\{A\}$. Si $A \rightarrow R_1$ ou $A \rightarrow R_2$, la décomposition est sans perte.

$A \rightarrow A, B, C$ (vérifier dans F).

Si A est clé, alors décomposition sans perte.

(b) $R_1(A, B, C)$ $R_2(C, D, E)$. Montrer que cette décomposition est avec perte d'information.

L'intersection est $\{C\}$. La décomposition est sans perte si $C \rightarrow R_1$ ou $C \rightarrow R_2$. Sinon, elle est avec perte.

Exercice n° 4

Pour répondre aux questions suivantes, vous pouvez utiliser le langage de programmation de votre prédilection. Je vous donne les réponses des premières questions, pour ne pas perdre trop de temps dans le choix des structures de données à utiliser. Elles sont d'écrites ci-après ¹ :

- un ensemble pour représenter une relation et chaque élément est une chaîne de caractères. L'exemple suivant est une d'éclaration de deux relations (tableau) constituées respectivement des 'éléments suivants $\{A, B, C, G, H, I\}$ et $\{X, Y\}$:

```
1 myrelations = [
2     {'A', 'B', 'C', 'G', 'H', 'I'},
3     {'X', 'Y'}
4 ]
```

- et les d'épendances est une liste de paire de deux ensembles. L'exemple suivant est une d'éclaration de l'ensemble des d'épendances suivantes $A \twoheadrightarrow B$; $A \twoheadrightarrow C$; $CG \twoheadrightarrow H$; $CG \twoheadrightarrow I$ et $B \twoheadrightarrow H$.

```
1 mydependencies = [
2     [{'A'}, {'B'}],           #A- >B
3     [{'A'}, {'C'}],           #A- >C
4     [{'C', 'G'}, {'H'}],       # CG - >H
5     [{'C', 'G'}, {'I'}],       # CG - >I
6     [{'B'}, {'H'}],           #B- >H
7 ]
```

1. Écrire une procédure qui permet de prendre en paramètre une liste de d'épendances fonctionnelles et les affiche.

```
1 def printDependencies ( F: "list of dependencies "):
2
3     for alpha , beta in F:
4         print("\t", alpha , " --> ", beta )
```

1. Quelque soit le langage de programmation que vous avez choisi, utiliser les m^emes structures de donn^ees

2. Ecrire une procédure qui permet de prendre en paramètre un ensemble de relations T et les affiche.

```

1 def printRelations ( T: "list of relations"):
2
3     for R in T:
4         print("\t", R)

```

3. Ecrire une fonction qui, prenant en paramètre un ensemble *inputset*, renvoie tous ces sous-ensembles. Rappelons qu'un sous-ensemble S d'un ensemble E est un ensemble tel que $S \subseteq E$. L'ensemble qui a pour éléments tous les sous-ensemble de E est noté $P(E)$. Et $card(P(E)) = 2^n$, où $n = card(E)$ (la cardinalité de E). N'oubliez pas l'instruction permettant d'importer *itertools*.

```

1 import itertools
2
3 def powerSet( inputset: "set"):
4     _result = []
5     for r in range (1, len ( inputset) +1):
6         _result += map ( set , itertools.combinations ( inputset , r))
7
8     return _result

```

4. Ecrire une fonction qui permet, étant donné un ensemble de dépendances fonctionnelles F et un ensemble d'attributs K , de retourner la fermeture (clôture) de K .
5. Ecrire une fonction qui permet, étant donné un ensemble de dépendances fonctionnelles F , de retourner la clôture de F . Rappel : la clôture de F est un ensemble constitué de toutes les dépendances fonctionnelles que l'on peut déduire de F .
6. Ecrire une fonction qui permet, étant donnée un ensemble de dépendances fonctionnelles F et deux ensembles d'attributs α et β , de retourner vrai si α détermine fonctionnellement β .
7. Ecrire une fonction qui permet, étant donnée un ensemble de dépendances fonctionnelles F , une relation R et un ensemble d'attributs K , de retourner vrai si K est une super-clé.
8. Ecrire une fonction qui permet, étant donnée un ensemble de dépendances fonctionnelles F , une relation R et un ensemble d'attributs K , de retourner vrai si K est une clé candidate.
9. Ecrire une fonction qui, étant donnée une relation R et un ensemble de dépendances fonctionnelles F , de retourner la liste de toutes les clés candidates.
10. Ecrire une fonction qui, étant donnée une relation R et un ensemble de dépendances fonctionnelles F , de retourner la liste de toutes les super-clés.
11. Ecrire une fonction qui permet, étant donnée un ensemble de dépendances fonctionnelles F et une relation R , de retourner une clé candidate.
12. Ecrire une fonction qui permet, étant donnée une relation R et un ensemble de dépendances fonctionnelles F , de retourner vrai si cette relation est en BCNF.
13. Ecrire une fonction qui permet, étant donnée un ensemble de relations T et une liste de dépendances fonctionnelles F , de retourner vrai si le schéma défini par ces relations est en BCNF.
14. Ecrire une fonction qui permet, étant donnée un ensemble de dépendances fonctionnelles F et un ensemble de relations T , d'implémenter l'algorithme de décomposition en BCNF, vu en cours.

```

(base) nourbenammar@Nours-MacBook-Air ~ % python3 /Users/nourbenammar/Downloads/exercice4_tp2.py
Dépendances :
{A} -> {B}
{A} -> {C}
{C, G} -> {H}
{C, G} -> {I}
{B} -> {H}

Relations :
{A, B, C, G, H, I}
{X, Y}

Sous-ensembles de {A,B,C} :
{A}
{B}
{C}
{A, B}
{A, C}
{B, C}
{A, B, C}

Fermeture de {A, G} :
{A, B, C, G, H, I}

```

Toutes les clés candidates de R :
{A, G}

Toutes les super-clés de R :

{A, G}
{A, B, G}
{A, C, G}
{A, G, H}
{A, G, I}
{A, B, C, G}
{A, B, G, H}
{A, B, G, I}
{A, C, G, H}
{A, C, G, I}
{A, G, H, I}
{A, B, C, G, H}
{A, B, C, G, I}
{A, B, G, H, I}
{A, C, G, H, I}
{A, B, C, G, H, I}

R est-elle en BCNF ? False

Décomposition BCNF de [R] :

{A, B}
{A, C}
{A, G, H, I}